Name – Sahil Patel

Date - 29/08/26

Sap_ID - 590037006

Subject - C PROGRAMMING

100 DAYS OF CODE - 4

(C PROGRAMMING)

❖ Problem Statement:

Correct the given C program, algorithm, and flowchart to perform multiplication of two integers without using the multiplication (*) operator. Implement the solution using repeated addition and ensure that it correctly handles positive numbers, negative numbers, and zero.

• Objective:

To implement multiplication of two integers using repeated addition instead of the multiplication (*) operator and verify the program using different test cases.

• Given Algorithm:

1) Define num1, num2, prod, index as integers.

2) Get num1, num2.

3) Set prod = 0, index = 1

4) Repeat steps 4.1 and 4.2 until (index <= num2)

- 4.1 prod = prod + num1
- 4.2 index = index + 1

5) Print prod

6) Stop

- **Here are the mistakes in the given algorithm:**

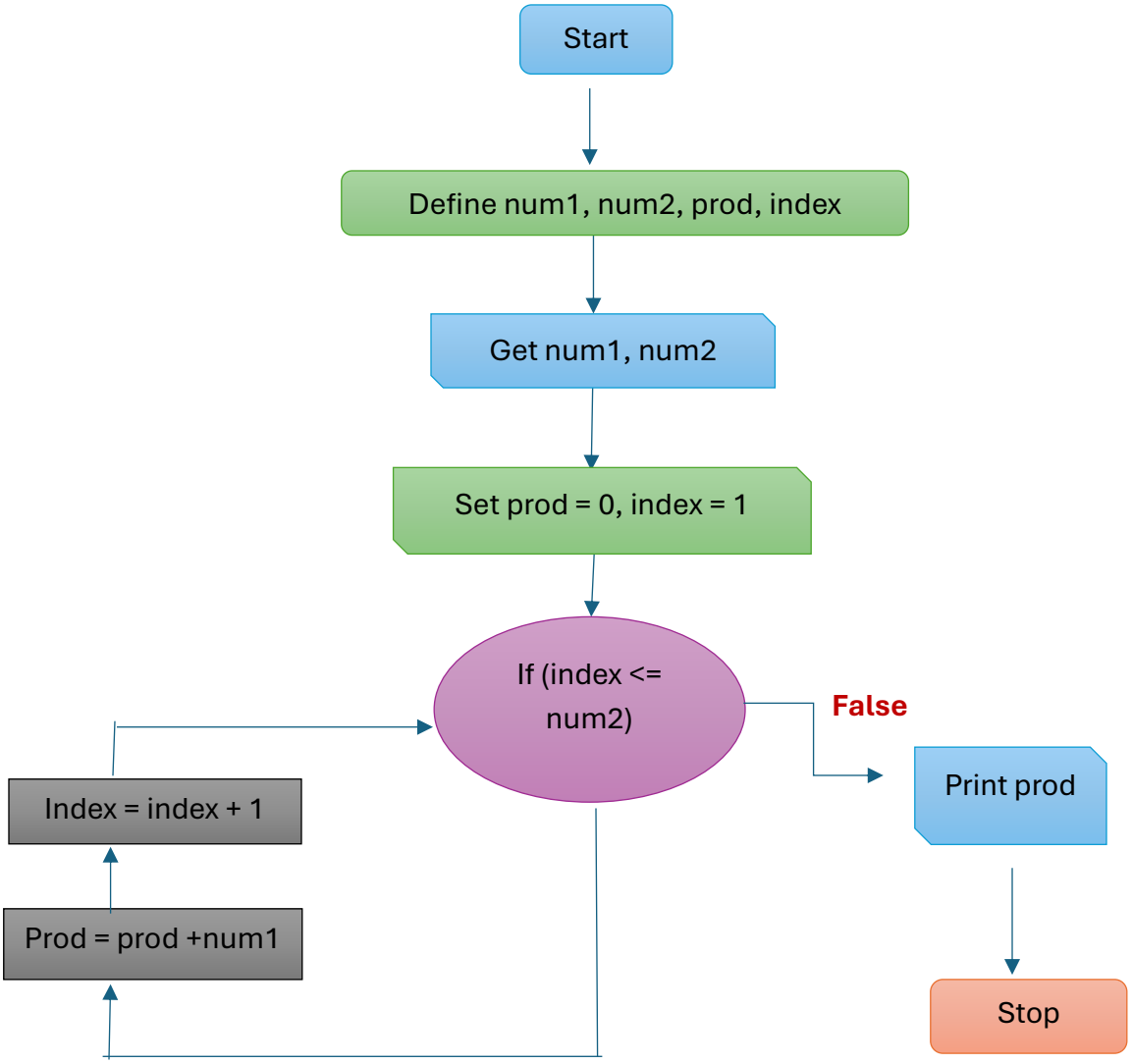
1) It works only for positive values of num2.

2) It does not work for negative numbers.

3)It does not correctly handle multiplication if one of the inputs is zero.

4)It does not determine the sign of the result.

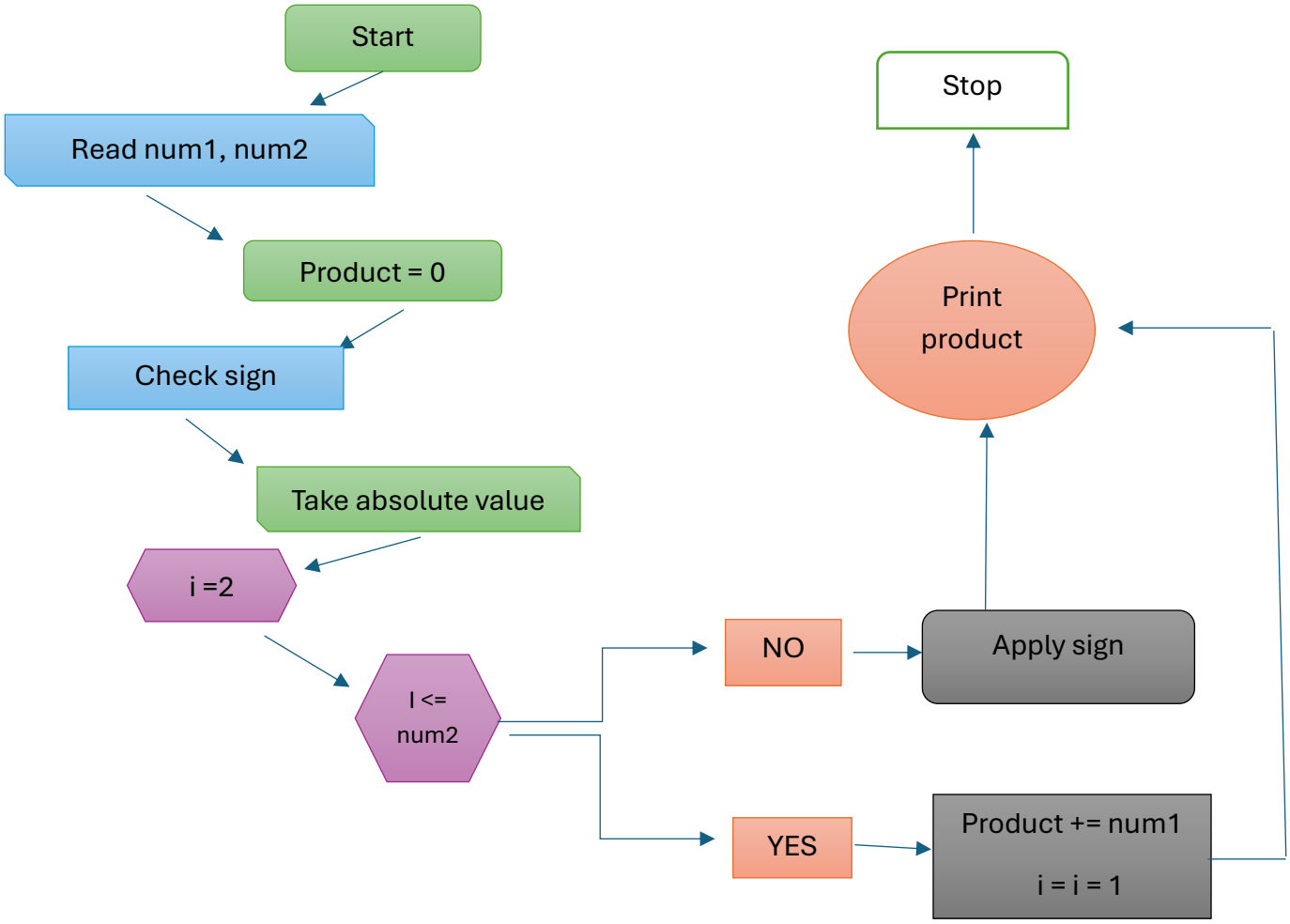
- **Flowchart**



- **Here is the correct algorithm:**

1. Start.
2. Read two integers num1 and num2.
3. Initialize product = 0.
4. Determine whether the final result should be negative.
5. Convert num1 and num2 to their absolute values.
6. Repeat num2 times:
    - product = product + num1
7. If the result should be negative, make product = -product.
8. Display the product.
9. Stop.

- **Here is the correct flowchart:**



❖ Corrected C Program

```
Session  Actions  Edit  View  Help
#include<stdio.h>
int main()
{
        int num1, num2;
        int product = 0;
        int i;
        int sign = 1;

        printf("Enter any two integers");
        scanf("%d %d", &num1, &num2);

        if(num2 < 0)
        {
                num1 = -num1;
                num2 = -num2;0
        }
        for(i = 1; i <= num2; i++)
        {
                product = product + num1;
        }
        printf("Product = %d", product)
        return 0;
}
~
```

- **TEST CASE 1)**

```
┌──(kali㉿kali)-[~/Desktop]
└─$ vi hdoc4.c

┌──(kali㉿kali)-[~/Desktop]
└─$ gcc hdoc4.c -o hdoc4

┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc4
Enter any two integers12 3
Product = 36
```

- **TEST CASE 2)**

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc4
Enter any two integers-6 5
Product = -30
```

- **TEST CASE 3)**

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc4
Enter any two integers -9 -4
Product = 36
```

- **TEST CASEE 4)**

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc4
Enter any two integers0 8
Product = 0
```

- **TEST CASE 5)**

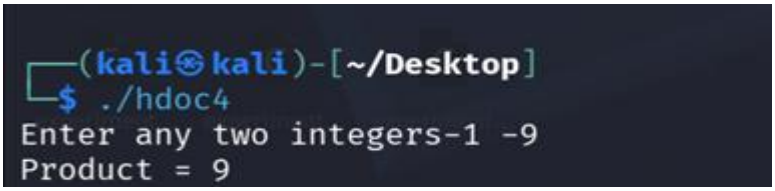
```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc4
Enter any two integers0 -7
Product = 0
```

- **TEST CASE 6)**

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc4
Enter any two integers0 0
Product = 0
```

- **TEST CASE 7)**

- **TEST CASE 8)**



❖ CONCLUSION:

This assignment demonstrated how we can multiply using repeated addition instead of using the * operator. The program was tested with positive numbers, negative numbers and zero and it produced correct outputs every time.